JavaScript Function call()

Back to JS page

Table of Contents

Using call() to Set this

The `call()` method calls a function with a given `this` value and arguments provided individually.

With `call()`, you can write a method once and then inherit it in another object, without rewriting the method:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function call()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using call() to Set this</h4>
<p id="demo"></p>

<script>
const person = {
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

const person1 = {
  firstName: "John",
  lastName: "Doe"
};

const person2 = {
  firstName: "Mary",
  lastName: "Smith"
};

document.getElementById("demo").innerHTML =
  person.fullName.call(person1) + "<br>" +
  person.fullName.call(person2);
</script>

</body>
</html>
```

Example 1

Result View Example

Borrowing a Method from Another Object

You can use `call()` to borrow a method from one object and use it for another object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function call()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Borrowing a Method from Another Object</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

const member = {
  firstName: "Jane",
  lastName: "Smith"
};

// Borrow the fullName method from person
document.getElementById("demo").innerHTML = person.fullName.call(member);
</script>

</body>
</html>
```

Example 2

Result View Example

The call() Method with Arguments

The `call()` method can accept arguments:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function call()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The call() Method with Arguments</h4>
<p id="demo"></p>

<script>
const person = {
  fullName: function(city, country) {
    return this.firstName + " " + this.lastName + ", " + city + ", " + country;
  }
};

const person1 = {
  firstName: "John",
  lastName: "Doe"
};

document.getElementById("demo").innerHTML = person.fullName.call(person1, "Oslo", "Norway");
</script>

</body>
</html>
```

Example 3

Result View Example

call() vs Normal Function Call

The difference between a normal function call and `call()` is that `call()` allows you to explicitly set what `this` refers to:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function call()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>call() vs Normal Function Call</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
function greet(greeting) {
  return greeting + ", " + this.name;
}

const user = {
  name: "Alice"
};

// Normal function call - "this" refers to window/global
// document.getElementById("demo1").innerHTML = greet("Hello"); // this.name would be undefined

// Using call() - "this" refers to user
document.getElementById("demo1").innerHTML = greet.call(user, "Hello");

const user2 = {
  name: "Bob"
};

document.getElementById("demo2").innerHTML = greet.call(user2, "Hi");
</script>

</body>
</html>
```

Example 4

Result View Example

Document

Document in project

You can Download PDF file.

Reference

- W3Schools JavaScript Function call()